

TECH

SECURITY

SECURITY

SECURITY

35 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК [owasp](#) / [mdn](#) / [общая практика](#)

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	XSS REACT XSS dangerouslySetInnerHTML Q1 · Q2 · Q3 · Q4 · Q5	XSS REACT XSS dangerouslySetInnerHTML
ДЕНЬ 2 СРЕДНИЙ	SANITIZATION ESCAPING CSP Q6 · Q7 · Q8 · Q9 · Q10 · Q11	SANITIZATION ESCAPING CSP
ДЕНЬ 3 СРЕДНИЙ	CSRF CORS PREFLIGHT Q12 · Q13 · Q14 · Q15 · Q16 · Q17	CSRF CORS PREFLIGHT
ДЕНЬ 4 ТЯЖЁЛЫЙ	TOKEN STORAGE localStorage vs cookies HttpOnly / SameSite Q18 · Q19 · Q20 · Q21 · Q22 · Q23	TOKEN STORAGE localStorage vs cookies HttpOnly / SameSite
ДЕНЬ 5 СРЕДНИЙ	LOGOUT PROTECTED ROUTES FE GUARD PERMISSIONS Q24 · Q25 · Q26 · Q27	LOGOUT PROTECTED ROUTES FE GUARD PERMISSIONS
ДЕНЬ 6 ТЯЖЁЛЫЙ	SECRETS IN BUNDLE LOGGING PII SUPPLY CHAIN Q28 · Q29 · Q30 · Q31	SECRETS IN BUNDLE LOGGING PII SUPPLY CHAIN
ДЕНЬ 7 СРЕДНИЙ	DEPS AUDIT SECURE DEFAULTS REVIEW Q32 · Q33 · Q34 · Q35	DEPS AUDIT SECURE DEFAULTS REVIEW

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ
ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1

НАГРУЗКА

СРЕДНИЙ

5 ВОПРОСОВ

XSS**REACT XSS****dangerouslySetInnerHTML**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

XSS REACT XSS dangerouslySetInnerHTML

ВОПРОСЫ ДНЯ

- Q01 Что такое XSS?
- Q02 Какие бывают виды XSS?
- Q03 Как React защищает от XSS по умолчанию?
- Q04 Когда React не защищает от XSS?
- Q05 Чем опасен dangerouslySetInnerHTML?

Что такое XSS?

ОПИСАНИЕ

Cross-Site Scripting – выполнение чужого JS в контексте вашей страницы. Источник – пользовательский ввод, попавший в DOM/HTML без санитизации. Результат – кража токенов, session hijack, defacement.

КАК РАБОТАЕТ

- 01 User input → HTML без escape.
- 02 Stored / Reflected / DOM-based.
- 03 Доступ ко всему (cookies, DOM, localStorage).
- 04 Лечение: sanitize / escape / CSP.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Public-facing apps.
- 02 User-generated content.

ПОДВОДНЫЕ КАМНИ

- 01 dangerouslySetInnerHTML без sanitize.
- 02 Inline JS в URL.

МОГУТ ДОСПРОСИТЬ

- 01 Какие виды XSS?
- 02 Чем CSP помогает?

ПОЛНАЯ СТАТЬЯ

<https://owasp.org/www-community/attacks/xss/>

ВОПРОС Q02

Какие бывают виды XSS?

ОПИСАНИЕ

Stored – script сохранён на сервере (комменты), показывается всем. Reflected – script в URL, выполняется при visit. DOM-based – JS на клиенте читает untrusted input и пишет в DOM.

КАК РАБОТАЕТ

- 01 Stored: script saved server-side.
- 02 Reflected: script в URL/POST.
- 03 DOM-based: client-side только.
- 04 Все три одинаково опасны.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Penetration tests.
- 02 Security audits.

ПОДВОДНЫЕ КАМНИ

- 01 DOM-based не видна server logs.
- 02 Reflected часто через email-phishing.

МОГУТ ДОСПРОСИТЬ

- 01 Чем DOM-based от reflected отличается?
- 02 Что такое self-XSS?

ПОЛНАЯ СТАТЬЯ

https://owasp.org/www-community/Types_of_Cross-Site_Scripting

Как React защищает от XSS по умолчанию?

ОПИСАНИЕ

JSX автоматически escapes `textContent` и `attribute-values`. `{userInput}` → escaped. JSX `attribute='X'` тоже escaped. Поэтому обычный React-код безопасен по умолчанию.

КАК РАБОТАЕТ

- 01 JSX auto-escape in `{expr}`.
- 02 Attributes escaped.
- 03 No template-injection.
- 04 Default-secure rendering.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой React-код.

ПОДВОДНЫЕ КАМНИ

- 01 `{}` только escapes string, не URL.
- 02 `React.createElement` также escapes.

МОГУТ ДОСПРОСИТЬ

- 01 Что НЕ escapes React?
- 02 Чем шаблонные движки опасны?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/javascript-in-jsx-with-curly-braces>

ВОПРОС Q04

Когда React не защищает от XSS?

ОПИСАНИЕ

`dangerouslySetInnerHTML`, `ref-DOM-manipulation` (`innerHTML`, `document.write`), URLs из `untrusted input` в `href/src` (`javascript:URL`), `eval/Function constructor`.

КАК РАБОТАЕТ

```
01 dangerouslySetInnerHTML.  
02 ref.current.innerHTML = ....  
03 href={'javascript:...'}.  
04 eval / new Function.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 User-content rendering.  
02 Markdown / WYSIWYG.  
03 Dynamic URLs.
```

ПОДВОДНЫЕ КАМНИ

```
01 javascript:URL атаки.  
02 Library, использующая dangerouslySetInnerHTML.
```

МОГУТ ДОСПРОСИТЬ

```
01 Что такое javascript-URL XSS?  
02 Чем DOMPurify помогает?
```

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react-dom/components/common#dangerously-setting-the-inner-html>

Чем опасен dangerouslySetInnerHTML?

ОПИСАНИЕ

Прямая инъекция HTML без санитизации = XSS. Если контент from-user – script выполнится. Применять только с DOMPurify.sanitize или для статического trusted-content.

КАК РАБОТАЕТ

- 01 Direct innerHTML injection.
- 02 No escape.
- 03 User-content без sanitize → XSS.
- 04 Always use DOMPurify.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Markdown render.
- 02 User comments.
- 03 Email preview.

ПОДВОДНЫЕ КАМНИ

- 01 Без sanitize – XSS.
- 02 Whitelist слишком широкий → onerror= attribute.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое DOMPurify?
- 02 Чем CSP помогает?

ПОЛНАЯ СТАТЬЯ

<https://github.com/cure53/DOMPurify>

ДЕНЬ 2

НАГРУЗКА

СРЕДНИЙ

6 ВОПРОСОВ

SANITIZATION**ESCAPING****CSP**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

SANITIZATION ESCAPING CSP

ВОПРОСЫ ДНЯ

- Q06 Как безопасно рендерить user-generated HTML?
- Q07 Что такое sanitization?
- Q08 Что такое escaping?
- Q09 Чем sanitization отличается от escaping?
- Q10 Что такое CSP?
- Q11 Как CSP помогает против XSS?

Как безопасно рендерить user-generated HTML?

ОПИСАНИЕ

DOMPurify.sanitize(html, options) – whitelist tags + attrs. Альтернативно – markdown-renderer с whitelist. Лучше всего – НЕ рендерить HTML, использовать text + markdown.

КАК РАБОТАЕТ

- 01 DOMPurify.sanitize.
- 02 Whitelist tags + attrs.
- 03 Markdown с whitelist (marked + DOMPurify).
- 04 Text rendering best.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Comments / posts.
- 02 Wiki-pages.
- 03 Email preview.

ПОДВОДНЫЕ КАМНИ

- 01 Whitelist слишком широкий.
- 02 Без CSP – second line of defense missing.

МОГУТ ДОСПРОСИТЬ

- 01 Чем react-markdown полезен?
- 02 Что такое CSP?

ПОЛНАЯ СТАТЬЯ

<https://github.com/cure53/DOMPurify>

ВОПРОС Q07

Что такое sanitization?

ОПИСАНИЕ

Очистка input от опасного контента. Удаляет / экранирует script-tags, on*-attributes, javascript:URLs. Whitelist подход – оставляет только safe. Tools: DOMPurify, sanitize-html.

КАК РАБОТАЕТ

- 01 Remove dangerous parts.
- 02 Whitelist allowed tags/attrs.
- 03 DOMPurify, sanitize-html.
- 04 Output ready to render.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Перед innerHTML / dangerouslySetInnerHTML.
- 02 Markdown rendering.

ПОДВОДНЫЕ КАМНИ

- 01 Blacklist опасный – что-то пропущено.
- 02 Sanitize только при render → store раздел.

МОГУТ ДОСПРОСИТЬ

- 01 Чем sanitize от escape отличается?
- 02 Где sanitize – клиент или сервер?

ПОЛНАЯ СТАТЬЯ

<https://github.com/cure53/DOMPurify>

Что такое escaping?

ОПИСАНИЕ

Конвертация спец-символов (<, >, &, ", ') в HTML-entities (<, >...). Не удаляет – делает безопасными для рендера. React делает в JSX автоматически. Не путать с sanitize.

КАК РАБОТАЕТ

- 01 < → <, > → >, & → &.
- 02 Auto в JSX.
- 03 Сохраняет содержимое, делает безопасным.
- 04 For text rendering.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Render text-content.
- 02 JSX expressions {}.

ПОДВОДНЫЕ КАМНИ

- 01 Escape для HTML ≠ escape для URL ≠ escape для JS.
- 02 Double-escape ломает.

МОГУТ ДОСПРОСИТЬ

- 01 Чем escape от sanitize отличается?
- 02 Что такое context-aware escape?

ПОЛНАЯ СТАТЬЯ

<https://owasp.org/www-community/attacks/xss/>

ВОПРОС Q09

Чем sanitization отличается от escaping?

ОПИСАНИЕ

Sanitize – удаляет опасное, сохраняет 'safe HTML'. Escape – конвертирует символы в entities, делая их text-content. Sanitize для HTML-render, escape для text-render.

КАК РАБОТАЕТ

- 01 Sanitize: filter dangerous, keep safe HTML.
- 02 Escape: convert symbols to entities.
- 03 Sanitize → render HTML.
- 04 Escape → render text.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 dangerouslySetInnerHTML → sanitize.
- 02 JSX text → escape (auto).

ПОДВОДНЫЕ КАМНИ

- 01 Wrong tool wrong context.
- 02 Double-processing.

МОГУТ ДОСПРОСИТЬ

- 01 Когда sanitize, когда escape?
- 02 Что такое context-aware?

ПОЛНАЯ СТАТЬЯ

<https://owasp.org/www-community/attacks/xss/>

Что такое CSP?

ОПИСАНИЕ

Content Security Policy – HTTP header, описывающий, откуда грузить ресурсы (scripts, styles, images). Блокирует inline-скрипты, eval, неразрешённые источники. Defense-in-depth против XSS.

КАК РАБОТАЕТ

- 01 HTTP header или meta-tag.
- 02 script-src / style-src / connect-src.
- 03 Block inline script / eval.
- 04 Nonce / hash для inline.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production apps.
- 02 High-security.

ПОДВОДНЫЕ КАМНИ

- 01 Strict CSP ломает inline scripts.
- 02 Без report-uri – нет visibility.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое nonce?
- 02 Чем strict-dynamic полезен?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP>

ВОПРОС Q11

Как CSP помогает против XSS?

ОПИСАНИЕ

Даже если XSS-script инъецирован, CSP может заблокировать его выполнение (если inline без nonce, или from неразрешённого origin). Defense-in-depth – после sanitize.

КАК РАБОТАЕТ

- 01 Block inline без nonce.
- 02 Whitelist origins для script.
- 03 Block eval / Function.
- 04 Report violations.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 High-stakes apps.
- 02 Compliance.

ПОДВОДНЫЕ КАМНИ

- 01 CSP сам по себе – не заменяет sanitize.
- 02 Слишком strict – ломает app.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое report-only?
- 02 Чем nonce от hash отличается?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/strict-csp>

CSRF**CORS****PREFLIGHT**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

CSRF CORS PREFLIGHT

ВОПРОСЫ ДНЯ

- Q12 Что такое CSRF?
- Q13 Когда CSRF актуален для frontend?
- Q14 Что такое CORS?
- Q15 Почему CORS не является auth?
- Q16 Что такое preflight request?
- Q17 Какие CORS-настройки опасны?

ВОПРОС Q12

Что такое CSRF?

ОПИСАНИЕ

Cross-Site Request Forgery – атака, заставляющая browser отправить request на ваш сервер с пользовательскими cookies. Пользователь не знал. Лечение: SameSite cookies, CSRF-tokens.

КАК РАБОТАЕТ

- 01 Browser auto-sends cookies.
- 02 Attacker site → POST на ваш API.
- 03 SameSite=Strict/Lax fixes most.
- 04 CSRF-token второй слой.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Form-based auth.
- 02 Cookie-based sessions.

ПОДВОДНЫЕ КАМНИ

- 01 Без SameSite = vulnerable.
- 02 JWT в Authorization header – CSRF не страшен.

МОГУТ ДОСПРОСИТЬ

- 01 Чем SameSite от CSRF-token отличается?
- 02 Когда CSRF не актуален?

ПОЛНАЯ СТАТЬЯ

<https://owasp.org/www-community/attacks/csrf>

Когда CSRF актуален для frontend?

ОПИСАНИЕ

При cookie-based auth (browser auto-sends). При SPA с JWT в Authorization header – CSRF не страшен (header не auto-sends). При Same-Origin sticky cookies.

КАК РАБОТАЕТ

- 01 Cookie-based session = vulnerable.
- 02 JWT в header = безопасно.
- 03 SPA в bearer-token = OK.
- 04 SameSite mitigates.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Legacy cookie-auth apps.
- 02 Server-rendered apps.

ПОДВОДНЫЕ КАМНИ

- 01 JWT in cookie = vulnerable если без CSRF-token.
- 02 JWT в localStorage → XSS-уязвим, не CSRF.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое double submit cookie?
- 02 Чем bearer от cookie отличается?

ПОЛНАЯ СТАТЬЯ

<https://owasp.org/www-community/attacks/csrf>

ВОПРОС Q14

Что такое CORS?

ОПИСАНИЕ

Cross-Origin Resource Sharing – protocol, контролирующий, может ли one origin делать requests к another. Server возвращает Access-Control-Allow-* headers. Browser enforces.

КАК РАБОТАЕТ

- 01 Browser enforces.
- 02 Server sends Allow-Origin headers.
- 03 Same-Origin: same scheme+host+port.
- 04 Preflight OPTIONS для non-simple.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 SPA на одном domain, API на другом.
- 02 Cross-domain integrations.

ПОДВОДНЫЕ КАМНИ

- 01 Wildcard * + credentials → не работает.
- 02 CORS не security feature – просто policy.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое preflight?
- 02 Чем simple от non-simple request отличается?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

Почему CORS не является auth?

ОПИСАНИЕ

CORS – browser-side policy, защищающий пользователя от cross-origin malicious sites. Не security для server – server должен сам проверять auth (cookies, tokens). CORS можно обойти из не-browser клиента.

КАК РАБОТАЕТ

- 01 Browser-only enforcement.
- 02 Server-side auth обязательна.
- 03 curl / postman ignore CORS.
- 04 CORS защищает user, не server.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Понимание боли когда CORS падает в проде.
- 02 Auth design.

ПОДВОДНЫЕ КАМНИ

- 01 CORS как auth – не работает.
- 02 Wildcard * без understanding – vulnerability.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое same-origin policy?
- 02 Чем CORS защищает?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

ВОПРОС Q16

Что такое preflight request?

ОПИСАНИЕ

OPTIONS-запрос, который browser делает перед non-simple cross-origin request (custom headers, methods кроме GET/POST/HEAD). Server должен ответить с Allow-* headers.

КАК РАБОТАЕТ

- 01 OPTIONS pre-CORS check.
- 02 Trigger: non-simple method/header.
- 03 Server: Access-Control-Allow-.*.
- 04 Cached через Access-Control-Max-Age.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой cross-origin app.

ПОДВОДНЫЕ КАМНИ

- 01 Server не отвечает OPTIONS → broken.
- 02 Slow preflight без caching.

МОГУТ ДОСПРОСИТЬ

- 01 Чем simple от non-simple отличается?
- 02 Что такое Allow-Headers?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS#preflighted_requests

Какие CORS-настройки опасны?

ОПИСАНИЕ

Access-Control-Allow-Origin: * + credentials (browser игнорирует, но опасно).
Allow-Origin: reflected без validation (echo back any origin). Allow-Methods: * для всех.

КАК РАБОТАЕТ

- 01 * + credentials = invalid.
- 02 Reflected origin без validation.
- 03 Allow-* для admin endpoints.
- 04 Allow-Headers: * exposes.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Security audits.

ПОДВОДНЫЕ КАМНИ

- 01 Reflected origin → CSRF возможна.
- 02 Слишком открытые → leakage.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое origin validation?
- 02 Чем Allow-Credentials от Allow-Origin отличается?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Access-Control-Allow-Origin>

ДЕНЬ 4

НАГРУЗКА

ТЯЖЁЛЫЙ

6 ВОПРОСОВ

TOKEN STORAGE localStorage vs cookies HttpOnly / Sa

ЧТО УЧИМ В ЭТОТ ДЕНЬ

TOKEN STORAGE localStorage vs cookies HttpOnly / SameSite

ВОПРОСЫ ДНЯ

- Q18 Где хранить access token?
- Q19 Где хранить refresh token?
- Q20 Какие риски у localStorage?
- Q21 Какие риски у cookies?
- Q22 Что такое HttpOnly cookie?
- Q23 Что такое SameSite cookie?

Где хранить access token?

ОПИСАНИЕ

In-memory (closure / store) для SPA – самое безопасное от XSS. localStorage – XSS-vulnerable. httpOnly cookie – недоступен JS, но требует CSRF-protection.

КАК РАБОТАЕТ

- 01 In-memory (best for XSS).
- 02 localStorage = XSS-vulnerable.
- 03 httpOnly cookie + CSRF-token.
- 04 Refresh из httpOnly, access in-memory.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 SPA auth.
- 02 Mobile-web.

ПОДВОДНЫЕ КАМНИ

- 01 LocalStorage – XSS читает.
- 02 Cookie без HttpOnly = JS читает.

МОГУТ ДОСПРОСИТЬ

- 01 Чем in-memory от localStorage отличается?
- 02 Что такое httpOnly?

ПОЛНАЯ СТАТЬЯ

https://owasp.org/www-project-cheat-sheets/cheatsheets/HTML5_Security_Cheat_Sheet#local-storage

ВОПРОС Q19

Где хранить refresh token?

ОПИСАНИЕ

httpOnly + Secure + SameSite=Strict cookie. Недоступен JS – XSS не украдёт. Browser auto-sends с requests на refresh endpoint. Long-lived (дни/недели).

КАК РАБОТАЕТ

- 01 httpOnly Secure SameSite=Strict.
- 02 Long-lived (days/weeks).
- 03 Auto-sent на refresh endpoint.
- 04 Inaccessible to JS.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 JWT-based auth.
- 02 SPA.

ПОДВОДНЫЕ КАМНИ

- 01 Refresh в localStorage = XSS-target.
- 02 Без SameSite – CSRF possible.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое SameSite=Strict?
- 02 Чем Secure от HttpOnly отличается?

ПОЛНАЯ СТАТЬЯ

https://owasp.org/www-project-cheat-sheets/cheatsheets/HTML5_Security_Cheat_Sheet

Какие риски у localStorage?

ОПИСАНИЕ

XSS-читает всё (5MB ваших данных). Не auto-cleanup на logout. Sync с кодом complex (storage event для tabs). Quota errors при overflow. JSON-сериализация.

КАК РАБОТАЕТ

- 01 XSS reads any localStorage.
- 02 No auto-cleanup.
- 03 Sync between tabs через storage event.
- 04 Quota exceeded errors.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Code review.
- 02 Threat modeling.

ПОДВОДНЫЕ КАМНИ

- 01 Token here = XSS-vulnerable.
- 02 Sensitive data leak.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое quota?
- 02 Чем sessionStorage отличается?

ПОЛНАЯ СТАТЬЯ

https://owasp.org/www-project-cheat-sheets/cheatsheets/HTML5_Security_Cheat_Sheet

ВОПРОС Q21

Какие риски у cookies?

ОПИСАНИЕ

Без HttpOnly – XSS читает. Без Secure – sniff over HTTP. Без SameSite – CSRF возможна. Limit 4KB. Auto-sent с every request – bandwidth overhead.

КАК РАБОТАЕТ

- 01 Without HttpOnly = XSS reads.
- 02 Without Secure = HTTP sniff.
- 03 Without SameSite = CSRF.
- 04 4KB limit, auto-sent.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Auth design.
- 02 Threat modeling.

ПОДВОДНЫЕ КАМНИ

- 01 Default cookies = vulnerable.
- 02 Слишком много cookies – bandwidth.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое SameSite-Lax?
- 02 Чем cookie scope полезен?

ПОЛНАЯ СТАТЬЯ

<https://datatracker.ietf.org/doc/html/rfc6265>

Что такое HttpOnly cookie?

ОПИСАНИЕ

Cookie с флагом HttpOnly недоступен JS (`document.cookie` не покажет). Browser отправляет автоматически с requests. Защищает от XSS-кражи.

КАК РАБОТАЕТ

- 01 Set-Cookie: token=abc; HttpOnly.
- 02 JS не может read.
- 03 Browser auto-sends.
- 04 XSS не украдёт.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Auth tokens (refresh / session).
- 02 Sensitive cookies.

ПОДВОДНЫЕ КАМНИ

- 01 JS-side checks (`isLoggedIn`) теперь сложнее.
- 02 Logout требует server-side cookie clear.

МОГУТ ДОСПРОСИТЬ

- 01 Чем Secure от HttpOnly отличается?
- 02 Как logout с HttpOnly?

ПОЛНАЯ СТАТЬЯ

<https://owasp.org/www-community/HttpOnly>

ВОПРОС Q23

Что такое SameSite cookie?

ОПИСАНИЕ

Атрибут cookie: Strict (только same-site requests), Lax (top-level navigation OK), None (cross-site allowed, требует Secure). Defense против CSRF.

КАК РАБОТАЕТ

- 01 Strict: only same-site.
- 02 Lax: top-level OK (default).
- 03 None: cross-site, requires Secure.
- 04 CSRF protection.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Auth cookies.
- 02 Session cookies.

ПОДВОДНЫЕ КАМНИ

- 01 None без Secure – invalid.
- 02 Strict ломает SSO redirect.

МОГУТ ДОСПРОСИТЬ

- 01 Чем Strict от Lax отличается?
- 02 Когда нужен None?

ПОЛНАЯ СТАТЬЯ

<https://datatracker.ietf.org/doc/html/draft-ietf-httpbis-rfc6265bis>

LOGOUT**PROTECTED ROUTES****FE GUARD****PERMISSIONS**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

LOGOUT

PROTECTED ROUTES

FE GUARD

PERMISSIONS

ВОПРОСЫ ДНЯ

Q24 Как обрабатывать `logout`?Q25 Как защищать `sensitive routes`?Q26 Почему `frontend auth guard` недостаточен?Q27 Как работать с `permissions` на `frontend`?

ВОПРОС Q 2 4

Как обрабатывать logout?

ОПИСАНИЕ

Server-side: invalidate session/refresh-token. Client-side: clear in-memory state, redirect /login, broadcast tabs (BroadcastChannel). Cookie cleanup (HttpOnly – server clear).

КАК РАБОТАЕТ

- 01 Server: invalidate session.
- 02 Client: clear store + redirect.
- 03 BroadcastChannel другим tabs.
- 04 Cookie clear (server-set with expired).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой auth-flow.

ПОДВОДНЫЕ КАМНИ

- 01 Только client-clear → token до expiry valid на server.
- 02 Tabs out of sync.

МОГУТ ДОСПРОСИТЬ

- 01 Чем BroadcastChannel помогает?
- 02 Что такое token revocation?

ПОЛНАЯ СТАТЬЯ

https://owasp.org/www-project-cheat-sheets/cheatsheets/Session_Management_Cheat_Sheet

Как защищать sensitive routes?

ОПИСАНИЕ

Frontend: ProtectedRoute с auth-check + redirect. Server-side: гарантированная проверка на каждом endpoint. Frontend – только UX, не security.

КАК РАБОТАЕТ

- 01 <ProtectedRoute>: check + redirect /login.
- 02 Server-side: auth middleware.
- 03 Permissions-check на каждом endpoint.
- 04 FE как UX, BE как security.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой auth-app.

ПОДВОДНЫЕ КАМНИ

- 01 Только FE-guard = security hole.
- 02 Race на initial load.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое route-guard?
- 02 Чем server-check от FE-check отличается?

ПОЛНАЯ СТАТЬЯ

https://owasp.org/www-project-top-ten/2017/A5_2017-Broken_Access_Control

ВОПРОС Q 2 6

Почему frontend auth guard недостаточен?

ОПИСАНИЕ

Frontend code публичный, любой может обойти (DevTools, curl, Postman). Real auth – на сервере. FE-guard улучшает UX, но не security.

КАК РАБОТАЕТ

- 01 FE code public.
- 02 Anyone can bypass через DevTools/curl.
- 03 Real auth = server-side.
- 04 FE = UX only.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Security audits.
- 02 Code review.

ПОДВОДНЫЕ КАМНИ

- 01 FE-only auth = security hole.
- 02 Trusting client.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое client-side trust?
- 02 Чем JWT verify от UI-check отличается?

ПОЛНАЯ СТАТЬЯ

https://owasp.org/www-project-top-ten/2017/A5_2017-Broken_Access_Control

Как работать с permissions на frontend?

ОПИСАНИЕ

Permission-list в auth-state. Hooks: useCan(action). Components: <Can action="...">. Все permissions из server-source. Server проверяет always – FE-check только для UX.

КАК РАБОТАЕТ

- 01 auth-store с permissions.
- 02 useCan / <Can>.
- 03 Server-source permissions.
- 04 Server-side enforcement.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 RBAC apps.
- 02 Multi-role tools.

ПОДВОДНЫЕ КАМНИ

- 01 Frontend-only check = security hole.
- 02 Stale permissions без relogin.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое RBAC vs ABAC?
- 02 Как обновлять permissions?

ПОЛНАЯ СТАТЬЯ

https://owasp.org/www-project-top-ten/2017/A5_2017-Broken_Access_Control

ДЕНЬ 6

НАГРУЗКА

ТЯЖЁЛЫЙ

4 ВОПРОСА

SECRETS IN BUNDLE**LOGGING PII****SUPPLY CHAIN**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

SECRETS IN BUNDLE

LOGGING PII

SUPPLY CHAIN

ВОПРОСЫ ДНЯ

- Q28 Как не раскрывать sensitive data в bundle?
- Q29 Какие данные нельзя логировать?
- Q30 Как защищать error reporting от утечки PII?
- Q31 Что такое supply chain attack?

Как не раскрывать sensitive data в bundle?

ОПИСАНИЕ

Никаких secrets в .env с публичным префиксом (VITE_ / NEXT_PUBLIC_). Server-side rendering для секретного. Source-maps в production только private (Sentry). Strings audit.

КАК РАБОТАЕТ

- 01 No secrets в public env.
- 02 Server-side для sensitive logic.
- 03 Sentry-uploaded sourcemaps только.
- 04 Audit strings перед deploy.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production deploys.
- 02 Compliance audits.

ПОДВОДНЫЕ КАМНИ

- 01 NEXT_PUBLIC_API_KEY – публично.
- 02 Source-maps + Sentry без access-control.

МОГУТ ДОСПРОСИТЬ

- 01 Чем server component защищает?
- 02 Что такое hidden sourcemaps?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/source-maps>

ВОПРОС Q29

Какие данные нельзя логировать?

ОПИСАНИЕ

PII: email, full name, phone, address, SSN, credit-card. Passwords. JWT/tokens.
Auth-headers. Sensitive financial. Health records (HIPAA).

КАК РАБОТАЕТ

- 01 PII / PHI / PCI.
- 02 Passwords / tokens / keys.
- 03 Auth-headers.
- 04 Financial details.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 GDPR / CCPA / HIPAA compliance.
- 02 Logging design.

ПОДВОДНЫЕ КАМНИ

- 01 Stack-trace может содержать.
- 02 Console.log в prod = leak.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое PII?
- 02 Чем PCI DSS требует?

ПОЛНАЯ СТАТЬЯ

<https://gdpr.eu/>

Как защищать error reporting от утечки PII?

ОПИСАНИЕ

Sentry beforeSend filter: redact emails/phones/PII. Whitelist полей. Не логировать request bodies sensitive endpoints. Strip URL-params с sensitive data.

КАК РАБОТАЕТ

- 01 Sentry beforeSend hook.
- 02 Whitelist fields.
- 03 Strip sensitive URL-params.
- 04 Test payloads на отсутствие PII.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production error tracking.
- 02 Compliance.

ПОДВОДНЫЕ КАМНИ

- 01 Default config – leaks PII.
- 02 Stack-trace не filtered.

МОГУТ ДОСПРОСИТЬ

- 01 Чем data-scrubbing полезен?
- 02 Как тестировать?

ПОЛНАЯ СТАТЬЯ

<https://docs.sentry.io/platforms/javascript/data-management/sensitive-data/>

ВОПРОС Q31

Что такое supply chain attack?

ОПИСАНИЕ

Атака через зависимости: npm-пакет (or его dependency) был compromised и содержит malicious код. Запущен в build / runtime. Жертва – все приложения, использующие пакет.

КАК РАБОТАЕТ

- 01 Malicious npm package.
- 02 Compromised maintainer account.
- 03 Typosquatting (right-pad vs leftpad).
- 04 Postinstall scripts opportunity.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 npm ecosystem всегда.

ПОДВОДНЫЕ КАМНИ

- 01 Всё, что в node_modules – потенциально untrusted.
- 02 Postinstall script может exfil secrets.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое typosquatting?
- 02 Чем lockfile защищает?

ПОЛНАЯ СТАТЬЯ

https://owasp.org/www-project-top-ten/2021/A06_2021-Vulnerable_and_Outdated_Components

DEPS**AUDIT****SECURE DEFAULTS****REVIEW**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

DEPS

AUDIT

SECURE DEFAULTS

REVIEW

ВОПРОСЫ ДНЯ

Q32 Как проверять npm dependencies?

Q33 Как работать с dependency audit?

Q34 Как проектировать secure defaults?

Q35 Какие security-рецензии нужны перед релизом fintech-фичи?

ВОПРОС Q32

Как проверять npm dependencies?

ОПИСАНИЕ

npm audit / yarn audit / pnpm audit – known vulnerabilities. Snyk / Socket для deeper analysis. Dependabot / Renovate для auto-PR. Lockfile review. Pin versions.

КАК РАБОТАЕТ

- 01 npm audit (CVE check).
- 02 Snyk / Socket.
- 03 Dependabot auto-PR.
- 04 Lockfile review.
- 05 Pin versions.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой npm-проект.
- 02 Compliance audit.

ПОДВОДНЫЕ КАМНИ

- 01 Audit без действий – false safe.
- 02 Auto-update без testing – break.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое npm audit?
- 02 Чем Socket от Snyk отличается?

ПОЛНАЯ СТАТЬЯ

<https://docs.npmjs.com/cli/v10/commands/npm-audit>

Как работать с dependency audit?

ОПИСАНИЕ

Регулярные runs (CI), severity filter (high/critical), prioritized fixes, security-related PR auto-merge с тестами, exception process для known false positives.

КАК РАБОТАЕТ

- 01 CI: npm audit --audit-level=high.
- 02 Prioritize critical first.
- 03 Auto-PR through Dependabot.
- 04 Exception для FP.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production projects.
- 02 SOC2 / compliance.

ПОДВОДНЫЕ КАМНИ

- 01 Игнор аудита → накопление.
- 02 Без exception process – каждый FP блокирует.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое CVSS?
- 02 Чем lockfile audit отличается?

ПОЛНАЯ СТАТЬЯ

<https://docs.github.com/en/code-security/dependabot>

ВОПРОС Q34

Как проектировать secure defaults?

ОПИСАНИЕ

Безопасное по умолчанию: HttpOnly+Secure+SameSite cookies, CSP включён, HTTPS only, sanitize on render, no eval, no innerHTML без sanitize, escape user input.

КАК РАБОТАЕТ

- 01 Secure-by-default cookie attrs.
- 02 CSP включён.
- 03 HTTPS-only.
- 04 Sanitize по умолчанию.
- 05 Lint rules.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Frameworks setup.
- 02 DS / starter-templates.

ПОДВОДНЫЕ КАМНИ

- 01 Defaults insecure → каждый разработчик решает.
- 02 Без enforcement – drift.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое secure-by-default?
- 02 Чем lint helps?

ПОЛНАЯ СТАТЬЯ

<https://owasp.org/www-project-secure-headers/>

Какие security-рецензии нужны перед релизом fintech-фичи?

ОПИСАНИЕ

Threat-modeling, dependency audit, source-map review, sensitive logging audit, auth-flow review, CSP-test, pen-test для critical, compliance-checklist (PCI / GDPR / SOC2).

КАК РАБОТАЕТ

- 01 Threat modeling.
- 02 Dep audit.
- 03 Sourcemap audit.
- 04 Logging audit.
- 05 Auth review.
- 06 Pen-test для critical.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Trading / payment.
- 02 Healthcare.
- 03 Financial.

ПОДВОДНЫЕ КАМНИ

- 01 Skip review под давлением.
- 02 Pen-test после release – поздно.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое threat modeling?
- 02 Чем pen-test от audit отличается?

ПОЛНАЯ СТАТЬЯ

<https://owasp.org/www-project-application-security-verification-standard/>

ФИНАЛЬНЫЙ ЧЕК - ЛИСТ

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ Описать XSS, виды, защиту React, dangerouslySetInnerHTML (Q1-Q5)
- ☐ Описать sanitize/escape/CSP (Q6-Q11)
- ☐ Описать CSRF, CORS, preflight (Q12-Q17)
- ☐ Описать token storage, localStorage vs cookie, HttpOnly/SameSite (Q18-Q23)
- ☐ Описать logout, protected routes, FE guard, permissions (Q24-Q27)
- ☐ Описать secrets в bundle, logging PII, supply chain (Q28-Q31)
- ☐ Описать deps audit, secure defaults, pre-release review (Q32-Q35)

EVENT LOOP / ASYNC МЕТОДИЧКА v1
МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru